UNIVERSIDAD NACIONAL JOSÉ MARÍA ARGUEDAS

Computación Gráfica · Unidad I · IIAD65 · Grupo 04

ASTRO-TRAZOS

Arquitectura, algoritmos y reproducción

Tipo de documento: Manual técnico
Versión: 20 de octubre de 2026
Repositorio: https://github.com/WILLIAMMDN/astro-trazos

Equipo consignado en el informe previo: Max William Medina Castro; Neil Angelo Hiyo Juarez Anderson; Allcca Mauycalla (nombre completo por confirmar).

Docente: Mtro. Jhamer Anderson Rojas Silva.

Andahuaylas, Perú · 2026

1. Plataforma y estructura

Aplicación cliente con HTML, CSS, SVG y JavaScript ES modules. Herramientas verificadas: Node 24.15.0, Vite 8.3.4, Vitest 5.0.5 y ESLint 10.13.0. No usa backend ni datos personales. La versión autónoma empaqueta los recursos para abrirse localmente.

Ruta	Responsabilidad
index.html	Escenas ilustradas y controles
src/playable.js	Entrada, estados, simulaciones y HUD
src/geometry/	Métricas, rutas y reconocimiento
src/math/Mat3.js	Transformaciones 2D homogéneas
src/clipping/CohenSutherland.js	Recorte al viewport
src/collision/Collisions.js	AABB e intersección
tests/	Pruebas automatizadas
scripts/make-standalone.mjs	HTML autónomo

2. Arquitectura y flujo gráfico

Arquitectura: navegador → eventos pointer → estado de escena → ajuste geométrico → evaluador de nivel → animación SVG → HUD. El menú, la escena activa, el modal de pausa y el resultado son estados de la interfaz. Los trazos se almacenan por escena y se pueden deshacer o borrar.

Pipeline de dibujo: (1) capturar coordenadas del puntero en el viewBox; (2) simplificar puntos y reconocer la intención geométrica; (3) cuantizar o ajustar cuando corresponde; (4) calcular longitud, ángulo, área o intersección; (5) recortar lo visible al viewport; (6) evaluar condiciones; (7) animar el resultado. La pista visual se dibuja en una capa separada y no altera el estado.



Figura 1. Escena de área en el juego.

3. Transformaciones y clipping

Mat3 representa matrices afines 3 × 3. La composición usada para Nilo es M = T(x,y)·R(θ)·S(sx,sy). Se aplica a un punto homogéneo $[x,y,1]^T$. El orden coloca al personaje, orienta su pose y ajusta su escala sin cambiar la geometría original del sprite.

Cohen–Sutherland asigna códigos de región izquierdo, derecho, superior e inferior. Si ambos extremos tienen código cero, acepta el segmento; si comparten un bit externo, lo rechaza; en otro caso, calcula la intersección con el borde y repite. Los caminos del pincel usan este recorte antes de formar su ruta SVG visible.

Archivos clave: src/math/Mat3.js, src/clipping/CohenSutherland.js y función clippedLinePath en src/playable.js. El viewBox es el sistema común para entrada y dibujo.

4. Colisiones y matemáticas jugables

El segmento $A(t)=p+t\cdot r$ y $B(u)=q+u\cdot s$ se cruza cuando $0\le t,u\le 1$. Se obtiene $t=(q-p)\times s/(r\times s)$ y $u=(q-p)\times r/(r\times s)$. Si $r\times s\approx 0$, el caso paralelo no produce cruce único. Una prueba AABB descarta segmentos separados antes de calcular t y u. El láser termina en la primera intersección encontrada.

Área: $A=b\cdot h$; la escena cuantiza dimensiones a 5 u para hacer construible el valor 1200 u². Vector: componentes $\Delta x,\Delta y$ y ángulo atan2. Rampa: inclinación $\text{atan2}(-\Delta y,\Delta x)$. Órbita: cierre del contorno y dispersión de radios. Polígono: número de lados y cierre. La lógica de cada escena está en src/playable.js; los cálculos reutilizables están en src/geometry/.



Figura 2. Láseres con colisiones entre segmentos.

5. Instalación y construcción

Clonar: git clone https://github.com/WILLIAMMDN/astro-trazos.git

Entrar a la carpeta: cd astro-trazos

Instalar dependencias reproducibles: npm ci

Servidor de desarrollo: npm run dev

Comprobaciones: npm run lint y npm test

Empaquetar demo: npm run standalone

Abrir dist/Astro-Trazos-Presentacion.html en Chrome o Edge.

No editar manualmente dist: es generado y está ignorado por Git. La carpeta 1_Codigo del ZIP incluye una copia lista para la exposición.

6. Pruebas, rendimiento y límites

Al 20 de octubre de 2026 se verificaron 17 pruebas Vitest aprobadas y ESLint sin errores. Se comprobó manualmente el arrastre de 40 u × 30 u en Chrome. La medición de FPS y la ejecución en los equipos concretos del Laboratorio 05 siguen pendientes; una cifra obtenida en un navegador automatizado no sustituye esa prueba.

Límites: las unidades dependen del espacio de escena, el reconocimiento aproxima trazos, y aún no hay evaluación controlada de aprendizaje. La adaptación a pantallas pequeñas y dispositivos antiguos requiere prueba adicional. No se recogen datos personales de niños en el juego.

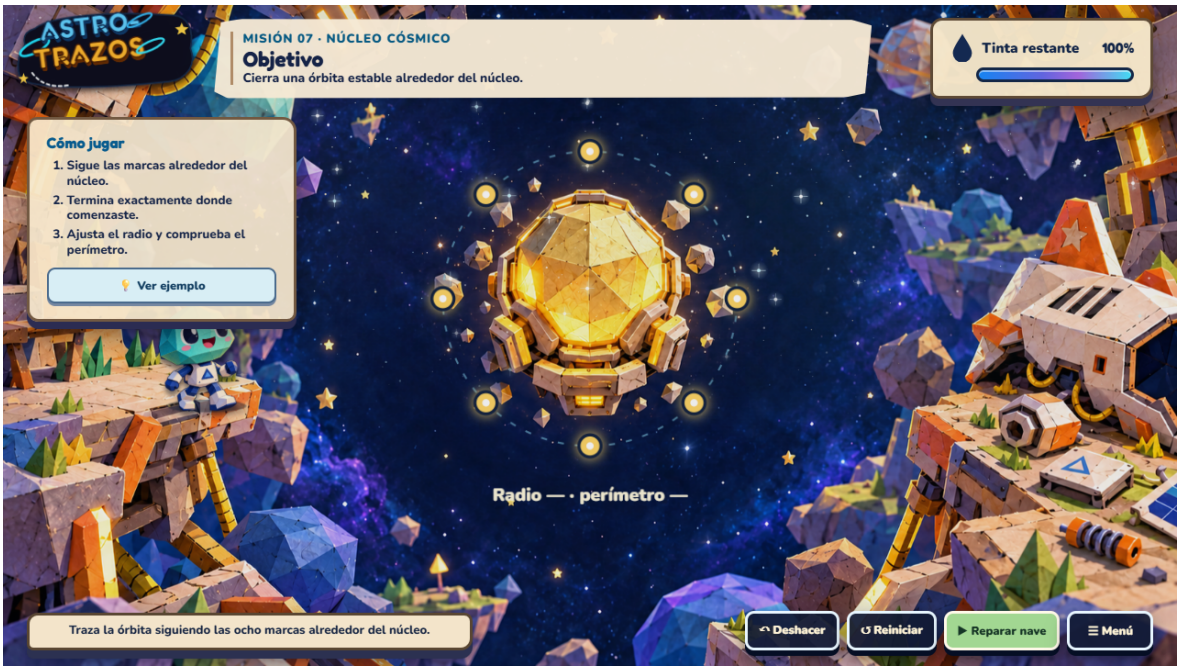


Figura 3. Escena orbital de la versión actual.

7. Mantenimiento y trazabilidad

Para agregar una escena: definir su SVG y objetivos, registrar estado y tinta en playable.js, implementar ajuste geométrico y evaluación, añadir al menú y crear pruebas de las condiciones matemáticas. Mantener el arte original y las fuentes en docs/ARTE_GENERADO.md y THIRD_PARTY_NOTICES.md. Revisar git log para contribuciones reales; no atribuir commits inexistentes.

Diagrama lógico: puntero → brushMath / shapeRecognition → evaluador → collision/Mat3/clipping → SVG + retroalimentación. Referencias de curso y fórmulas constan en el informe técnico.